

IDEA-GP: Instruction-Driven Architecture with Efficient Online Workload Allocation for Geometric Perception

Suquan Zhang
Tsinghua University
Beijing, China
zhangsq23@mails.tsinghua.edu.cn

Yu Hu
Tsinghua University
Beijing, China
huy24@mails.tsinghua.edu.cn

Yunfei Xiang
Tsinghua University
Beijing, China
xiang-yf22@mails.tsinghua.edu.cn

Dawei Zhao
Tsinghua University
Beijing, China
zhaodawei@tsinghua.edu.cn

Yuanfan Xu
Tsinghua University
Beijing, China
xuyf20@mails.tsinghua.edu.cn

Qingmin Liao
Tsinghua University
Beijing, China
liaoqm@tsinghua.edu.cn

Jincheng Yu
Tsinghua University
Beijing, China
yu-jc@mail.tsinghua.edu.cn

Yu Wang
Tsinghua University
Beijing, China
yu-wang@mail.tsinghua.edu.cn

Abstract

The algorithmic complexity of robotic systems presents significant challenges to achieving generalized acceleration in robot applications. On the one hand, the diversity of operators and computational flows within similar task categories prevents the reuse of specialized computational units. On the other hand, task variations and environmental dynamics can cause workload fluctuations, leading to inefficient resource utilization.

This paper focuses on the geometric perception capability of robots, taking localization and mapping as the basic applications, and proposes IDEA-GP, an Instruction-Driven Architecture with Efficient online workload Allocation for Geometric Perception. Built around an array of general computational units designed for spatial positioning representations, IDEA-GP supports a wide range of robot pose-related computational tasks. IDEA-GP employs a compiler to perform online workload analysis and resource allocation. It generates instructions tailored to processing elements (PEs) to schedule computations, thereby accelerating optimization problems and enhancing geometric perception performance. Deployed on the ZCU102 evaluation board, IDEA-GP demonstrates an average speedup of 7.5 $\times$ over the Intel CPU and 19.7 $\times$ over the ARM CPU in Simultaneous Localization and Mapping (SLAM) tasks, and a 16.4 $\times$ speedup over the Intel CPU and 41.6 $\times$ over the ARM CPU in Structure from Motion (SfM) tasks.

Keywords

Robot, Instruction Set Architecture, Hardware Accelerator, Geometric Perception, Optimization

ACM Reference Format:

Suquan Zhang, Yu Hu, Yunfei Xiang, Dawei Zhao, Yuanfan Xu, Qingmin Liao, Jincheng Yu, and Yu Wang. 2025. IDEA-GP: Instruction-Driven Architecture with Efficient Online Workload Allocation for Geometric Perception. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*, June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3695053.3731080>

1 Introduction

A robot is a highly complex system that integrates a variety of technologies, each with numerous algorithmic implementations [48]. This complexity makes it challenging to strike a balance between generality and efficiency in hardware design [47]. On the one hand, dedicated hardware accelerators often experience a decline in efficiency under certain task conditions and are generally unable to perform computations for variant tasks [54]. On the other hand, general hardware accelerators, which are designed to ensure computational efficiency, may require reconfiguration to accommodate the requirements of specific tasks [17]. Failure to optimize hardware for a specific task can lead to substantial computational waste, thereby reducing overall processing efficiency.

Environmental perception is a critical aspect of robot. The ability to comprehend geometric environments, including essential capabilities such as localization and mapping, is indispensable for the vast majority of robot applications [26].

The advancement of artificial intelligence and the emergence of embodied intelligence have further emphasized the need for environmental perception in robots. With advances in artificial intelligence and the emergence of embodied intelligence, robots increasingly need to extract information from large volumes of visual data, placing higher demands on their computational capabilities. Furthermore, the requirements for robots' scene understanding capabilities are shifting from two-dimensional (2D) to three-dimensional (3D) environments, accompanied by an increasing demand for higher precision mapping [40]. Consequently, these advancements and requirements have substantially heightened the computational complexity of geometric perception tasks. The



This work is licensed under a Creative Commons Attribution 4.0 International License. *ISCA '25, Tokyo, Japan*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1261-6/25/06
<https://doi.org/10.1145/3695053.3731080>

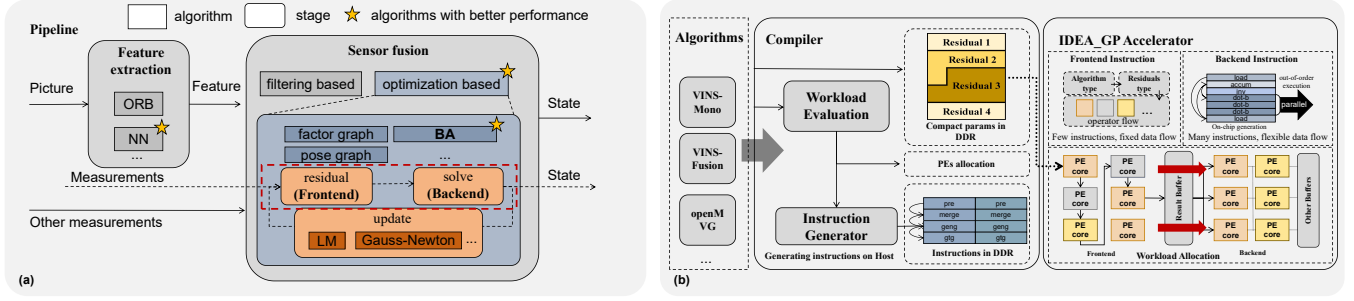


Figure 1: (a) The pipeline of geometric perception algorithms. We focus on the optimization problem solving, and these algorithms share a common computational framework, namely iterative computation of residuals and sparse equation solving. (b) The workflow of IDEA-GP. The architecture performs data rearrangement, PEs allocation, and instruction generation through a compiler, while the accelerator schedules computations for the front end and back end based on the output from the compiler.

geometric perception tasks we discuss mainly include optimization-based visual-SLAM (vSLAM) and SfM tasks.

Geometric environmental perception for robots needs optimization based on the robot’s pose and sensor data across various time frames [21, 44, 50]. Due to the substantial volume of visual feature point information in each frame, the optimization process entails solving and constructing an exceptionally large system of linear equations [25, 36]. This complexity often becomes a bottleneck in the system, resulting in considerable time consumption. Traditional methods are limited to achieving a frequency of 5Hz [11]. Therefore, leveraging the sparsity of visual information to effectively address the optimization problem is a critical issue. Consequently, a significant number of studies focus on accelerating the resolution of these tasks [4, 5, 10, 14, 15, 33, 56].

Among these studies, some research efforts expedite the process by designing dedicated accelerators tailored for specific algorithms [26]. These studies typically achieve high levels of performance. However, due to their specialized patterns designed for a singular computational flow, they often lack generality. Minor variations in the problem can sometimes cause the accelerator ineffective, preventing computational units from being used for different tasks.

Another issue is that when confronted with various task scenarios, the workload during problem construction is significantly unbalanced. Hardware must dynamically allocate resources according to the specific tasks to prevent computational waste.

The limitations of generality in robot optimization accelerators stem from two primary challenges. Firstly, **the formulation and solution of optimization problems involve a variety of operators**, such as different types of residuals, derivative operations, and equation-solving techniques. These elements generate distinct computational flows depending on the specific problem structure, which complicates the ability to support diverse tasks using fixed hardware.

Secondly, when confronted with different task scenarios, **the construction of problems leads to significantly varying workloads** [17]. For example, the computational load involved in the formulation and resolution of optimization problems can differ

markedly across various tasks. The computational ratio of the Frontend to the Backend is 1:1 in VINS [38], while it reaches 3:1 in OpenMVG [28]. To prevent unnecessary computational waste, hardware must efficiently allocate resources online in accordance with the varying workloads.

Motivated by the aforementioned issues and challenges, this paper proposes IDEA-GP, an Instruction-Driven Architecture for Geometric Perception with Efficient Online Workload Allocation. IDEA-GP employs instructions to schedule a series of unified basic computational units, facilitating the execution of various tasks. The computations within these tasks are streamlined to be performed through combinations of operators supported by these fundamental computational units.

In IDEA-GP, the Processing Elements (PEs) are managed through instructions, enabling the compiler to dynamically allocate an optimal number of PEs to various tasks for efficient workload distribution, without the need to regenerate hardware. Additionally, this architecture is scalable. In response to varying performance demands, we can adjust the computational capacity by deploying arrays of PEs of different sizes. The workflow of IDEA-GP is shown in Figure 1.(b). In practical tasks, the compiler first assesses the resource consumption based on the task type and problem formulation, and then configures the hardware accordingly. Following this, it generates control instructions, which are transmitted to the hardware to initiate computation.

We deploy our architecture on the Zynq UltraScale+ ZCU102 and evaluate its performance for geometric perception tasks, including SLAM and SfM. For SLAM tasks, we evaluate the optimization solutions of VINS-Mono [38] and VINS-Fusion [37] on several trajectories from the EuRoC dataset [6]. For SfM tasks, we use OpenMVG [28] for optimization on the SceauxCastle and DJI Mini2 datasets [28].

In summary, this paper makes the following contributions:

- We design a general PE architecture. By analyzing the spatial geometric representation and key operators in robot pose calculations, this PE supports a variety of computations related to robot poses. Through the scheduling of data among PEs, our architecture handles the majority of operations required for solving optimization problems. Moreover, due to

the unified construction of the PEs, our architecture is scalable, allowing for the adjustment of computational power by varying the number of PE cores.

- We analyze the data flow in optimization problems and, based on this analysis, propose a PE-centric instruction set architecture designed to streamline data processing. By generating instructions on-chip, the architecture imposes minimal bandwidth overhead. Additionally, with support for out-of-order and parallel execution, the basic instructions deliver high computational efficiency.
- We develop a compiler for geometric perception problems. It assesses workloads across different tasks and generates optimal resource configurations for the hardware online. Subsequently, the compiler generates control instructions based on this configuration. We also conduct evaluations on tasks such as SfM and SLAM. These evaluations reveal the variation in computational loads within different tasks. Due to our efficient online workload allocation and instruction execution, the 24-core architecture deployed on the Zynq UltraScale+ ZCU102 achieves an average speedup of 7.5× over the Intel CPU and 19.7× over the ARM CPU in SLAM tasks, and a 16.4× speedup over the Intel CPU and 41.6× over the ARM CPU in SfM tasks.

2 Background

2.1 Robot Geometric Perception

Robot environmental comprehension is primarily evident in their self-localization and structural understanding of the world. In practical task scenarios, the processes of robot positioning and mapping are closely intertwined. On the one hand, robots integrate sensor data into a global coordinate system based on their pose to create a map. On the other hand, they use feature information from the map to enhance their positioning accuracy.

In this process, if localization and mapping are performed concurrently during the robot's motion, it is referred to as an SLAM [7] task. In contrast, tasks that do not have stringent requirements for real-time performance and temporal consistency of observations are known as SfM tasks. SfM [40] can be employed as the frontend processing module for 3D reconstruction techniques like 3D Gaussian Splatting (3DGS), but often faces real-time performance limitations due to its requirement for higher-precision map construction.

In various SLAM algorithms, visual-SLAM [46], which relies on visual information, involves substantial computational demands due to the sparsity of visual features. However, it also presents significant opportunities for optimization. The geometric perception tasks we discuss mainly include visual-SLAM (vSLAM) and SfM tasks.

As illustrated in Figure 1.(a), from an algorithmic pipeline perspective, various SLAM and SfM methods share a common framework. Different types of SLAM receive and process distinct sensor data before fusing them. For instance, visual information must be processed to extract features before it can be integrated with other types of data.

Generally, the Frontend and Backend of SLAM refer to feature extraction and sensor fusion, respectively. However, in this paper,

we specifically refer to the Frontend and Backend as the optimization stages of residual calculation and sparse equation solving, as shown in the Figure 1.(a).

In the feature extraction stage, neural network algorithms have already demonstrated excellent performance, and neural network accelerators have also developed significantly. Therefore, IDEA-GP focuses on accelerating sensor fusion.

In the sensor fusion stage, optimization-based methods can achieve better results than filtering methods, and this class of algorithms shares the same optimization framework. Specifically, they iteratively compute residuals and solve sparse equations to obtain the optimal estimation. We refer to these two components as the Frontend and Backend of the optimization algorithm.

Different algorithms primarily differ in the composition of residuals and the methods used to solve sparse equations. For example, bundle adjustment (BA) optimization involves more visual residuals and uses feature point positions as optimization variables, which results in a larger sparse matrix. Pose graph optimization typically adds residuals related to loop closure. Factor graph optimization often solves sparse equations based on maximum a posteriori (MAP) estimation. These differences essentially lie in constructing different types of sparse equations. As shown in Figure 2, where the ultimate goal is to solve the equation $A\Delta x = b$.

Given our focus on addressing issues related to the sparsity of visual data, we adopt the BA method, which commonly employs the Schur complement for solving equations in the Backend. This approach is particularly well-suited for handling the challenges posed by sparse visual data.

2.2 The General Algorithm Description

Figure 2 illustrates the construction of residuals and equations in practical scenarios. Here, x denotes the camera pose, while y represents the positions of the observed feature points.

2.2.1 Frontend Equation Construction. From Figure 2, it can be seen that to construct an optimization problem, a variety of loss functions are first needed. These functions, known as residuals in optimization, represent the differences between observed and estimated values [12]. The purpose of solving the optimization is to minimize this error and obtain the optimal estimate for the optimization variables.

We use x to denote all optimization variables (such as position of the robot mentioned in Section 2.1), and f to denote the Residuals. For example, as shown in Figure 2, x comprises the camera poses and feature point poses, with $x = [y_i, x_{i-1}, x_i]$. The Residuals are composed of visual reprojection residuals and IMU residuals, represented by f_{rep} and f_{imu} . Here, $f = [f_{rep}, f_{imu}]$. Both f and x are vectors. The optimization problem can then be formulated as:

$$\min_x F(x) = \min_x \frac{1}{2} \|f(x)\|^2 \quad (1)$$

This is a least squares problem, which we solve iteratively using the Gauss-Newton method. By performing a first-order Taylor expansion on $f(x)$, we obtain:

$$\min_x F(x + \Delta x) \approx \min_x \frac{1}{2} \|f(x) + J(x)\Delta x\|^2 \quad (2)$$

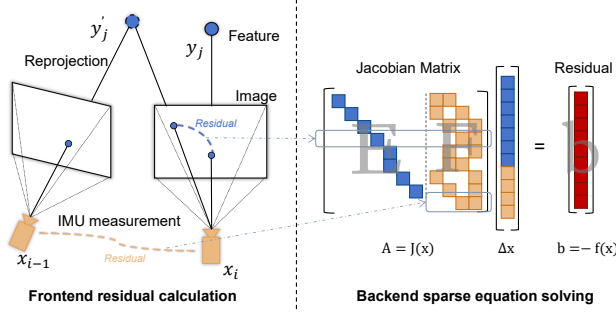


Figure 2: Construct the Residuals and Jacobian matrices based on the task scenario. Corresponding to the Frontend and Backend of the optimization problem.

In this equation, $J(x)$ represents the Jacobian matrix of the residuals. When $F(x)$ has a minimum, the derivative of $F(x + \Delta x)$ with respect to Δx is zero, which gives us:

$$J(x)^T J(x) \Delta x = -J(x)^T f(x) \quad (3)$$

We can formulate the equation as $J(x) \Delta x = -f(x)$, has a clear physical meaning. where $J(x)$ is the Jacobian matrix of the residuals $f(x)$ with respect to the variables x , and Δx represents the update to the variables x .

In the Frontend process, it aims to obtaining Jacobian Matrix and loss function to solve the update to the variables. As shown in Figure 2, the problem is reduced to constructing a system of linear equations in the form $A \Delta x = b$, in which A represents Jacobian matrix and b refers to residuals calculated by loss function. In each iteration, we need to solve for a set of derivatives and residuals to form the matrix A and the vector b .

2.2.2 Backend Equation Construction. In the context of visual scenes, A is a very sparse matrix. To handle this issue, it is necessary to simplify A . By employing the Schur Elimination method, we can reduce the matrix to a smaller dense matrix S of size $n \times n$.

Schur elimination requires dividing the independent variables into two parts, as shown in Figure 2. For the visual residual part, we can consider the poses of the visual feature points and the camera poses as two separate parts, $x = [x_{feature}, x_{camera}]$. Let the matrix of the Jacobian related to the feature points be denoted as matrix E , and the matrix related to the camera poses be denoted as matrix F . We have:

$$\begin{aligned} S &= F^T F - F^T E (E^T E)^{-1} E^T F \\ &= F^T F - G E^T F \end{aligned} \quad (4)$$

$$\begin{aligned} r &= F^T b - F^T E (E^T E)^{-1} E^T b \\ &= F^T b - G E^T b \end{aligned} \quad (5)$$

In which $G = F^T E (E^T E)^{-1}$. Thus, the equation $A \Delta x = b$ can be transformed into $S x_{camera} = r$, where x_{camera} (camera poses) is a part of optimization variables vector, allowing us to quickly obtain an estimate of the camera pose. Afterwards, based on the optimized camera pose, we can solve for the pose estimation of the feature points.

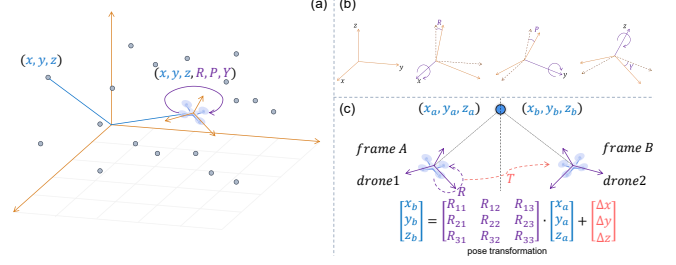


Figure 3: Representation of object poses in space and conversion formula for relative poses.

As we mentioned in Section 2.1, SLAM focus more on pose of the robot and SfM places relatively more emphasis on the feature points. Due to the small scale of x_{camera} , especially in SfM problems, the length of $x_{feature}$ is at least 100 times that of x_{camera} . The main computational effort in the Backend part lies in the solution of the Schur complement for the matrices S and r .

3 Workflow

IDEA-GP use unified Processing Elements to support the vast majority of calculations for optimization problems. Each PE has its unique index, which allows us to freely allocate the workload of different tasks. Figure 1.(b) illustrates the workflow of IDEA-GP.

For the Backend computation of optimization problems, we schedule data through instructions. These instructions achieve different computational flows by assigning computational tasks to units with specified indices, ensuring an equitable distribution of the workload.

In the Frontend, for the same problem, the composition of residuals is the same. The compiler only needs a small number of instructions to fix the computational pattern in the accelerator and statically schedule all data flows.

The allocation of PEs for the front and back ends of IDEA-GP is adjusted according to different tasks. Our compiler has a built-in workload analysis module that provides a rough estimate of the workload for both the front and back ends based on the current task type and the categories of residuals entered by the user. With this estimated computational load, we can configure the number of PEs on the hardware accelerator side and generate instructions accordingly.

4 General Kernel for Pose Transformation

4.1 Robot Spatial Geometric Representation

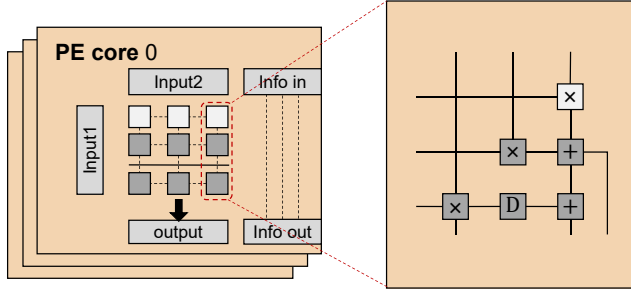
The basic geometric representation of robots is derived from the representation methods of particles and rigid bodies in physics.

As shown in Figure 3, the pose of an object is represented by two components, position and orientation, which together form a 6-dimensional space; meanwhile, the map is represented by a set of points in space.

In fact, the state of a rigid body can be fully represented by a 6-dimensional pose. However, the Euler angle representation has some issues: It requires the target to be rotated sequentially around the x , y , and z axes in a specific order, as shown in Figure 3.(b).

Table 1: Primitive Operation Types

Operator	Description
RR	Rotation matrix multiplication
RV	Rotation matrix-vector multiplication
VP	Vector addition
NR	Scalar multiplication of a matrix
NV	Scalar multiplication
$(\cdot)^\wedge$	Skew-symmetric matrix

**Figure 4: We design the PE to support operations in Table 1. In this figure, D is a data delay unit constructed using a small buffer.**

Otherwise, different results will be obtained. Moreover, Euler angles can lead to gimbal lock, preventing smooth interpolation over the surface of a sphere.

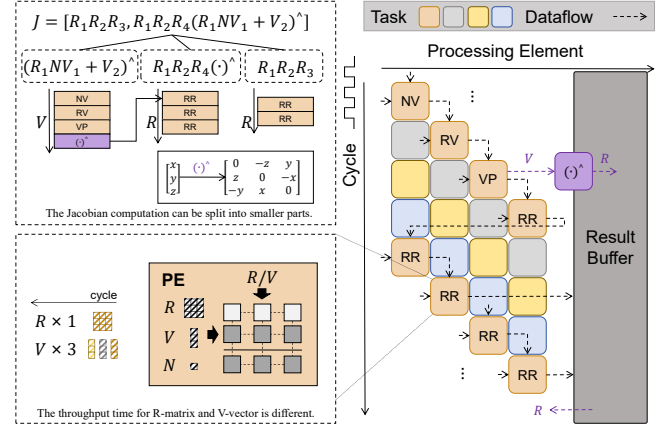
To address these issues, a common method in the field of robot is to use a 3×3 rotation matrix to represent the orientation. By left-multiplying the position with a rotation matrix, rotations around the origin can be achieved. In robot tasks, observed pose data typically comes directly from sensors. This data is then transformed to determine the pose in other coordinate frames. For example, in Figure 3, the same feature point is observed by two drones. The rotational relationship between Frame A of Drone 1 and Frame B of Drone 2 can be represented by R . The translation of the coordinate system is represented by $T = [\Delta x, \Delta y, \Delta z]$. Thus, the transformation of the feature point between different frames can be represented by the rotation R and the translation T between.

In other words, the kinematic transformations associated with robot poses can invariably be constructed from the multiplication and addition operations between a 3×3 rotation matrix R and a 3×1 translation vector T .

4.2 General PE Design

Based on Section 4.1, we can infer a crucial insight: the computations for both the Frontend and Backend in the optimization problem can be realized through a combination of simple operators such as multiplication and addition involving 3×3 matrices and 3×1 vectors. These operations constitute our set of basic operators.

Firstly, since the robot residuals are always composed of the difference between the observed pose and the estimated pose, and both of these poses are obtained through pose transformations, the residual operations in the Frontend can always be solved by basic pose transformations. Secondly, for the computation of the

**Figure 5: By splitting the Frontend computation, we can achieve a relatively fixed computation flow.**

Jacobian, it can be demonstrated that the partial derivatives of the residuals with respect to position and orientation can be transformed into derivatives with respect to a small orientation perturbation, and further reduced to a series of basic operations involving rotation matrices and skew-symmetric matrices to complete the calculation[43].

In solving the Backend equations, the pose representation of the robot directly constrains the size of the optimization variables. These variables are always concatenated from multiple 3D vectors, with each vector representing a position or orientation. This leads to non-zero blocks in the Jacobian matrix consistently appearing in an $n \times 3m$ format, where n is the dimension of an individual residual, and m is the dimension of the optimization variables. Hence, we can always decompose a large matrix multiplication into several 3×3 sub-matrix multiplications, which are then added into the result matrix. The skew-symmetric matrix operation expands a 3-dimensional vector into a 3×3 matrix through data rearrangement [43].

In summary, by supporting the set of basic operators listed in Table 1, we can accomplish both the Frontend and Backend computations within the optimization algorithm.

Targeting these operators, we design our unified PE, as depicted in Figure 4. By utilizing a simple 3×3 matrix array, we calculate the various operators listed in Table 1. Since all operations can be decomposed into small matrix multiplications, our matrix multiplication does not account for larger matrix multiplications; hence, the use of Multiply-Accumulate (MAC) units is unnecessary. Instead, directly using separate multipliers and adders can efficiently conduct operations and achieve the same outcome. By replacing accumulators with adders, we save resources.

Moreover, during the computation process, the accompanying auxiliary information flows with the data, including the target location of the data and the index information used for RAM addresses generation. This flow of information enables us to use the PE as a core unit for streamlining instruction processing, effectively allowing instructions to flow with the operations.

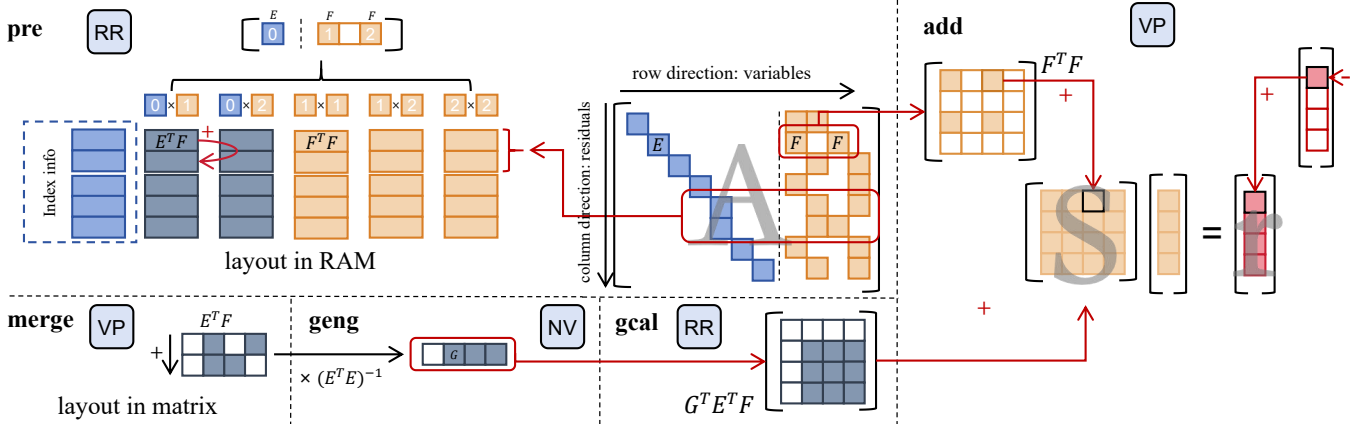


Figure 6: Dataflow of the Backend in optimization problems

5 PE Support for Frontend Computing

To facilitate the computation of Jacobian matrices and residuals in the front end, we design the computational pattern illustrated in Figure 5. This design is based on two key observations derived from our analysis of Frontend calculations.

Firstly, the computation of the entire Jacobian matrix can be decomposed into calculations on smaller blocks, which can be represented by the operators in Table 1, as illustrated in the top-left corner of Figure 5. A single J-matrix can be represented by two 3×3 matrices, which are derived from fundamental operations. In the formula, R represents the rotation matrix, V signifies the vector, and N represents the scalar. Furthermore, the primary operands of these operations are consistently R or V . Notably, without involving skew-symmetric matrix operations, R retains its identity as a matrix, and V remains a vector after being processed by the operators outlined in Table 1. This indicates that by partitioning the computational flow with skew-symmetric matrices, the primary subjects of computation in each segment remain unchanged.

Secondly, despite the variety of residual forms, the types of residuals utilized within a single task are limited. This suggests that a substantial amount of data will undergo repetitive operations, which can be easily pipelined.

The scheduling pattern illustrated in Figure 5 facilitates the pipelined computation of each block by interconnecting the inputs and outputs of the PEs. Given that the throughput time for R and V in the PE varies, enhancing PE utilization and ensuring pipeline alignment requires that the number of residual tasks being processed as V is three times greater than when R is in flow. However, operations involving skew-symmetric matrix operations increase the data scale, necessitating the storage of certain data in the result buffer. This data is stored according to its source from various residuals.

With the expansion of the PE scale, read and write operations may lead to data access conflicts. To mitigate these issues, we spatially separate memory read and write operations over time. Furthermore, to address unavoidable conflicts, we incorporate matrix multiplications with identity matrices I into our computations to defer read and write actions, thereby preventing conflicts.

6 Backend Dataflow and IDEA-GP ISA

Since the calculation flow for specific residuals is fixed, we employ a pre-designed scheduling method to automatically manage computations on the Frontend. To support the elimination of sparse matrices for various Backend tasks, this section outlines the computational procedures for Jacobian matrices of different structures, resulting in a unified execution process made up of basic operations as listed in Table 1. These execution flows are integrated into our instruction set.

The IDEA-GP ISA achieves high-throughput streaming processing through PE-centric operations. Additionally, we augment the computational efficiency of the architecture through the design of data storage access patterns that facilitate computational unit access and by employing efficient instruction execution techniques.

6.1 Unified Execution Flow

For different types of optimization problems, their primary impact on elimination is manifested in the compositional ratios and quantities of various residuals, as well as the number and form of the optimization variables.

In the Jacobian matrix formulated for robot optimization problems, each non-zero entry carries a specific physical implication. The columns of the Jacobian matrix are aligned with various residuals, whereas the rows correspond to distinct independent variables.

In the column direction, different types of residuals occupy different numbers of rows. Thus, we can partition the matrix along the column direction, processing one matrix block corresponding to each residual at a time. This is depicted in Figure 6, where each block of matrix A in the column direction matches the dimensions of a single residual. These operations are independent when there are no inter-residual dependencies, such as covisibility relationships.

In the row direction, various types of residuals are linked to optimization variables of differing sizes and counts. This is reflected in matrix A , where there may be a different number of blocks in the row direction, but the dimension of each block in the row direction is always a multiple of 3 (the dimension of the

Table 2: IDEA-GP ISA

Instruction	Description	Operands
load Bd Last TYPE	load data to Bram	Bd:destination, Last:last load of pre, TYPE=mission type
load-i Rd Cs Len	load data to DR according to CR	Rd:destination, Cs:source, Len:length
dot-b Pi Bd Bs TYPE	calculation of 3×3 dot	Pi:PE index, Bd:destination, Bs:source, TYPE=s/l
dot-bi Pi Bd Cs TYPE	calculation of dot according to CR	Pi:PE index, Bd:destination, Cs:source, TYPE=s/l
dot-r Pi Bd Rs TYPE	calculation of dot according to FR	Pi:PE index, Bd:destination, Rs:source, TYPE=s/l
accum Bs Bd Last	accumulate blocks	Bd:destination, Bs:source, Last:last accum of merge
add Bd Bs	add blocks	Bd:destination, Bs:source
inv Bs TYPE	Calculating the inverse of matrix	Bs:source, TYPE:matrix size
pre Pi Bd Len Last TYPE	generate instructions for <i>pre</i> stage	Pi:PE index, Bd:destination, Len:length, Last:last pre, TYPE=mission type
merge Bs Last TYPE Code	generate instructions for <i>merge</i> stage	Bs:source, Last:last merge, TYPE=s/l, Code:position code
geng Pi Len TYPE	generate instructions for <i>geng</i> stage	Pi:PE index Len:length TYPE=s/l
gcal Pi Len TYPE	generate instructions for <i>gcal</i> stage	Pi:PE index Len:length TYPE=s/l
read Bd Rs Len	read data from DDR to BRAM	Bd:destination, Rs:source Len:length
write Bs Wd Len	write data from BRAM to DDR	Bd:destination, Rs:source Len:length

robot's pose is a multiple of 3). We can always use the operators from Table 1 to carry out computations on these blocks.

Consequently, the matrix operations between large sparse matrices are converted into operations on individual small blocks, which are supported by the operators listed in Figure 6 and are independent for each set of unrelated residuals. For each group of residuals, the computation process can be broken down into five distinct stages:

As shown in Figure 6, there are five operational stages for each row:

1. **pre**: Conduct matrix operations among the blocks within a single row. For the computation of the S matrix, this involves calculations of $(E^T E)^{-1}$, $E^T F$, and $F^T F$. The outcomes of these computations are stored in the Bram in the format illustrated in Figure 6.

2. **merge**: Based on the correlations between different residuals, merge terms. For visual terms, such correlations stem from co-visible points.

3. **geng**: For each $E^T F$, calculate the $G = (E^T F)^T (E^T E)^{-1}$ and store it in Bram.

4. **gcal**: Calculate the $GE^T F$ matrix based on the obtained G .

5. **add**: Add the calculated $F^T F$ and $GE^T F$ to the final result matrix S .

The first two stages determine all intermediate results within a set of relevant information, while stages 3 and 4 further process this information. Stage 5, in fact, occurs before each update of the S matrix.

6.2 IDEA-GP ISA DESIGN

6.2.1 Register Definition. In our architecture, RAM is primarily used for storing intermediate results. Apart from that, IDEA-GP comprises two types of registers: **Data Registers (DR)** and **Configuration Registers (CR)**. These two types of registers are used to fetch partial data from RAM for caching and computation.

For instance, at the *gcal* stage, the computed n G matrices will undergo $n + C(2, n)$ matrix multiplications with each other. Therefore, it is necessary to preload the data from the RAM into the DR to facilitate task distribution on the PEs.

The Configuration Register (CR) is mainly used to store the address of various $E^T F$ blocks in RAM after the *merge* stage is completed.

6.2.2 Basic Instruction Type. For the five stages of the elimination data flow, we propose seven types of basic instructions. These basic instructions include load, read, write, acc, dot, add, and inv. The ISA definition is shown in Table 2. Each instruction is 32 bits in length. Corresponding to the aforementioned five stages, the *pre* stage is implemented through load, dot, and inv instructions, the *merge* stage is implemented through acc, and geng and gcal are implemented through variants of the dot instruction and load instructions. The read and write instructions handle memory scheduling between our architecture and DDR.

In addition to the basic instructions, there are four high-level instructions: pre, merge, geng, and gcal. The CPU sends these high-level instructions to control the hardware for computation, and these instructions are decoded on-chip into a series of basic instructions as shown in Figure 7. This method significantly reduces the bandwidth requirements for instructions. These high-level instructions do not align perfectly with the first four stages of the elimination data flow. For example, the tasks of the add stage are actually split between the pre and gcal instructions, and a single pre instruction may encompass the *pre* stage of multiple rows of residuals. The use of these high-level instructions also aims to facilitate further load scheduling on the PEs, to balance the workload across different PEs.

For various types of tasks, we can control the computation flow by adjusting the TYPE field. The TYPE field in the advanced instruction pre is used to modify tasks during the *pre* stage. Different TYPE fields generate different instructions on-chip, while the TYPE fields in other instructions primarily indicate the input dimension of one operand when the PE processes data. The TYPE=s/l, where s and l represent 1×3 and 3×3 matrix inputs, corresponding to the RV and RR operators in Table 1.

6.2.3 Efficient Execution. We utilize two techniques to enhance our instruction execution efficiency: **out-of-order execution** and **parallel execution**.

Firstly, regarding out-of-order execution, there is a clear dependency between our high-level instructions; the computation of the subsequent stage must begin only after the completion of the previous stage. To minimize the waste of computational resources, if the next instruction has no dependency with the current one, we will prioritize dispatching instructions that have no dependencies, the addresses of dependency-constrained instructions flow alongside

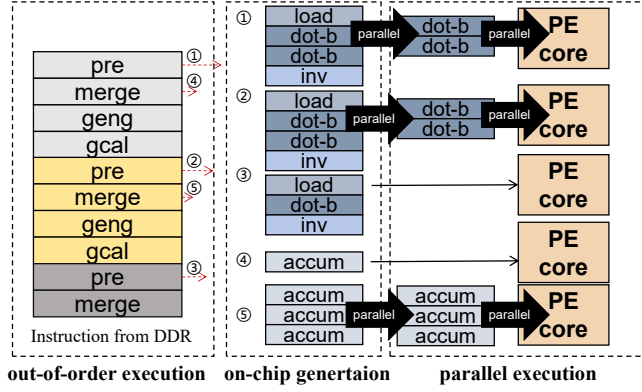


Figure 7: We reduce the pressure on the instruction bandwidth and enhance the execution efficiency by generating instructions on-chip, out-of-order execution, and parallel execution methods.

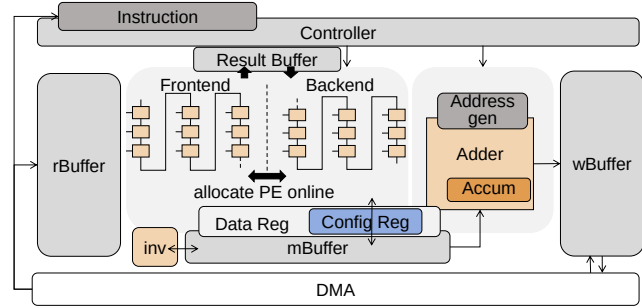


Figure 8: IDEA-GP hardware architecture.

the data from the preceding instruction and are skipped during execution, as shown in Figure 7.

Once the data computation is finished, the address also flows out simultaneously. This address directs the controller to jump to the corresponding instruction. This approach not only makes our architecture more conducive to pipelining, but also allows the accelerator to leverage the outflow of these addresses to identify the next instruction to execute. This enables out-of-order execution and effectively resolves instruction dependency issues.

For parallel execution, the basic instructions generated on-chip often involve many similar operations. For instance, during the *gcal* stage, the same matrix multiplications are performed on the data in the DR. These similar instructions are executed in parallel, loading the tasks into the PEs simultaneously.

7 Hardware Design

Figure 8 shows the hardware architecture of IDEA-GP. Computational data and instructions are transferred from DDR to the chip via DMA. This data is stored in two locations: the read Buffer (rBuffer) for accessing data and the Instruction Memory for the controller. The computation module consists of several components: an PEs array, an adders array, and specialized operation units such as skew-symmetric matrix and inversion units.

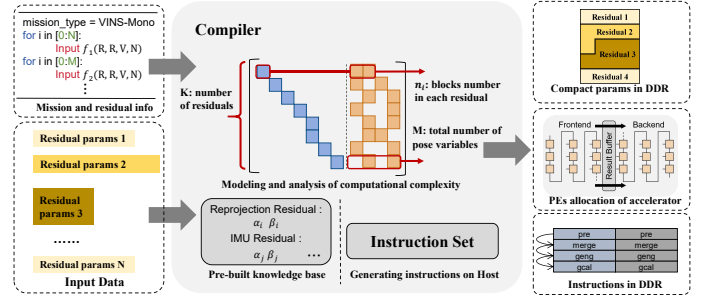


Figure 9: Proposed architecture of the compiler.

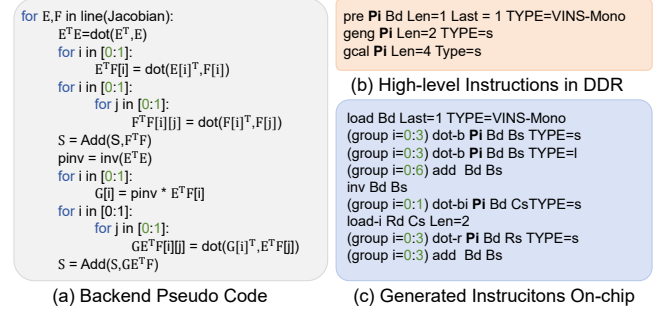


Figure 10: Example of computation using IDEA-GP ISA.

The computation units consist of several parts: an PEs array, an adders array, and specialized operation units such as skew-symmetric matrix and inversion units.

The PEs, which constitute the largest portion, are spatially divided into two sections for performing Frontend and Backend computations. As each PE is connected to both the Result Buffer and the middle Buffer (mBuffer), we can activate the PEs through instructions, thereby allocating a different number of PEs to the front and back ends for online workload allocation.

In IDEA-GP, most storage modules are organized based on the residual terms of the data source. For example, a row in the rBuffer contains variables for the same residual term, and a row in the Result Buffer corresponds to a row of the Jacobian, which represents a single residual. However, to boost the parallel processing capability of the PEs, a row in the mBuffer stores a row from several intermediate result matrices. In the wBuffer (write Buffer), a row stores a 6×6 matrix block from the *S* matrix, as pose variables typically manifest in a 6-dimensional format. To determine the final storage location of each block of data, the index information of the data within the Jacobian matrix flows with it through the PE, and is ultimately decoded into an address in the wBuffer.

8 Compiler Design

To efficiently adapt to various application scenarios, we design a compiler centered around the proposed general-purpose PEs. The workflow of the compiler is illustrated in Fig. 9. We implemented our compiler in C++, which functions as the hardware driver.

The compiler accepts several inputs, including the residual type of the current application task, optimization variables for visual

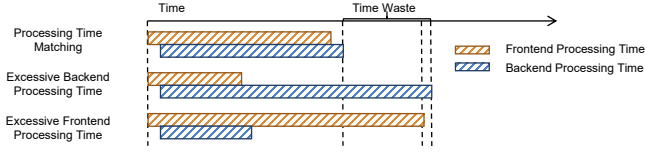


Figure 11: Optimal computation speed in a pipelined architecture is achieved by balancing the workloads of the Frontend and Backend on the PEs.

residuals, and other relevant observational parameters. By analyzing these inputs, the compiler firstly arranges data into the DDR in a tightly packed format for efficient PE access. Second, it dynamically generates an optimal PE resource configuration scheme based on the type and structure of each residual, improving processing efficiency across diverse tasks. Finally, it configures the parameters for the instructions and writes them into the pre-allocated physical memory addresses of the computer to drive the accelerator.

Different application scenarios result in varying computational loads, often leading to imbalanced demands between the Frontend and Backend phases. To address this, the compiler analyzes the input workload and produces a PE resource configuration scheme.

The workload for the Residual and Jacobian calculations in the Frontend phase is modeled as:

$$\#Jacobian = \sum \alpha_i \times k_i \quad (6)$$

where α_i denotes the estimated computational cost for the i -th residual type, and k_i represents the count of the i -th residual type. A pre-built knowledge base stores the computational costs for all residual patterns.

The Backend workload is modeled as:

$$\#Schur = \sum \beta_i \times k_i \times n_i^2 \quad (7)$$

Here, n_i is the number of non-zero matrix blocks in one row of the Jacobian matrix for the i -th residual type. Since each Jacobian row undergoes transpose and multiplication operations, the computational cost is estimated as $O(n_i^2)$. Varying block sizes for different residual types are weighted by β_i .

After predicting workloads for both the Frontend and Backend, PE resources are allocated as follows:

$$\frac{Col(Frontend)}{Col(Backend)} = \frac{\#Jacobian}{\#Schur} \quad (8)$$

where $Col(Frontend)$ and $Col(Backend)$ indicate the number of PE columns allocated to the respective modules. This maximizes the efficiency of the production line, as shown in Figure 11.

This allocation framework enables adaptive hardware resource configuration, ensuring efficient performance across tasks.

As shown in Fig. 10, an example conversion from pseudocode in the Backend phase to on-chip instructions is illustrated. The process begins with workload modeling (Fig. 10.(a)), followed by generating high-level instructions stored in the DDR (Fig. 10.(b)). Finally, basic instructions (Fig. 10.(c)) are generated on-chip in real time to control the Backend PEs.

Table 3: Segmented Workload Testing on EuRoC dataset [6]

Scenario	Segment A	Segment B	Segment C
#Residual num	1951	818	2737
Frontend time (ms)	2.227	0.880	3.281
Backend time (ms)	1.633	0.774	2.095
Rate	1.36	1.14	1.57

9 Evaluation

In this section, we evaluate our hardware architecture by selecting VINS-Mono, VINS-Fusion, and OpenMVG as representative cases for SLAM and SfM tasks. Furthermore, we assess several widely used datasets, including the EuRoC drone dataset [6] and the Sceaux Castle SfM dataset [28], to enhance the validity of our experimental results. Our architecture is implemented on the Zynq UltraScale+ ZCU102 platform and operates at a frequency of 166.7 MHz. All results presented in this section pertaining to the Intel CPU were obtained using the 13th Generation Intel® Core™ i7-13650HX processor, which operates at a frequency of 2.60 GHz. Furthermore, the results associated with the ARM CPU are derived from the Arm Cortex®-A78AE 2 GHz processor. Given the sparsity of the data, the GPU offers only limited acceleration. Our tests indicate that the computation time on the GPU differs by no more than 10% compared to that on the CPU. Similar results are noted in an issue¹, which aligns with our findings. As such, we omit the GPU-related results in this context.

In Section 9.1, we present experimental validation that demonstrates the varying load conditions associated with different tasks. Subsequently, our architecture is configured with 24 PEs for the experiments conducted in this study. The rationale for this configuration is discussed in Section 9.3. Section 9.2 assesses the accuracy of our compiler in evaluating workloads. Section 9.4 elucidates that the instruction-driven approach does not lead to excessive bandwidth consumption. Finally, Section 9.5 provides a summary of the resource utilization and timing report derived from our FPGA deployment.

9.1 Workload in Different Tasks

In this section, we perform evaluations on the workloads associated with various tasks. Figure 12 illustrates the computation time results for various tasks performed on the Intel CPU. Figure 12.(a) and Figure 12.(b) respectively depict the proportions of Frontend and Backend processing times for the VINS-Mono and VINS-Fusion algorithms across different trajectories within the EuRoC dataset [6]. It is noteworthy that the ratio of Frontend to Backend computation times for the same algorithm remains relatively stable, indicating that variations of the dataset exert minimal influence on this ratio.

During segmented tests on the same trajectory, extreme conditions cause fluctuations in the number of visual features and the ratio of co-visible points, which impact the workload distribution. For example, Table 3 shows the computation times for the Frontend and Backend across three segments (A, B, and C) from the MH_04 trajectory in the EuRoC dataset [6]. We measured the average computation times for the Frontend and Backend during each optimization cycle, with each segment lasting one second. The

¹<https://github.com/ceres-solver/ceres-solver/issues/759>

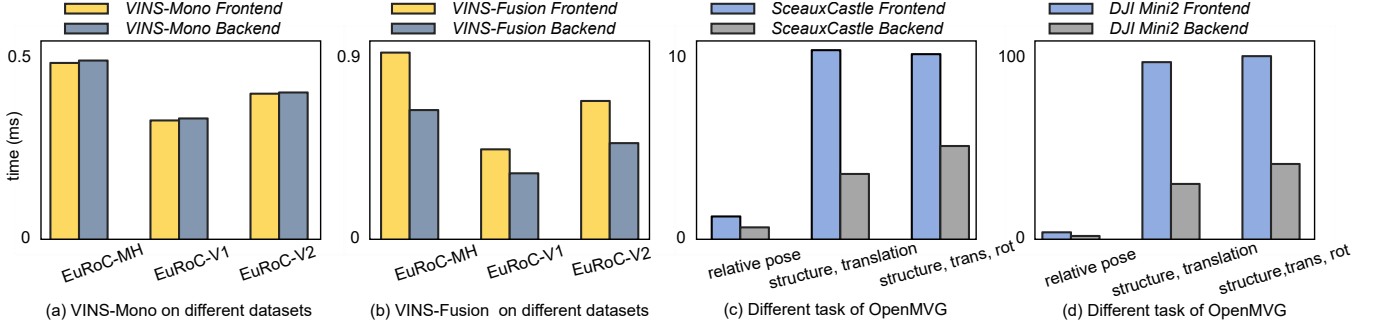


Figure 12: The CPU computation times suggest that workload variations depend on the task category.

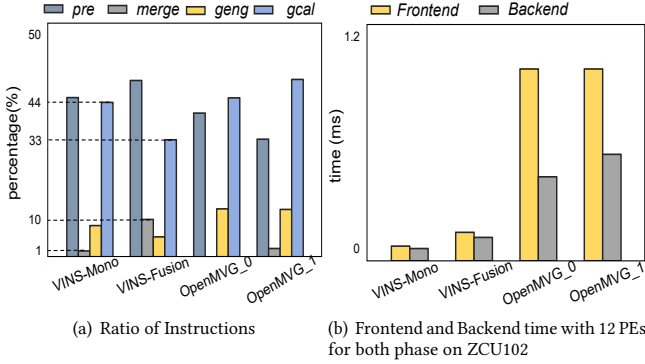


Figure 13: Equal resource allocation between Frontend and Backend on ZCU102 results in analyzed computation ratios and times across stages.

dataset starting at 127.3 seconds, and the corresponding rosbag times for the segments are as follows: 133.1 to 134.2 seconds for Segment A, 150.8 to 151.8 seconds for Segment B, and 187.6 to 188.8 seconds for Segment C.

The computation times for the Frontend and Backend vary in accordance with the number of visual residuals. Specifically, since our hardware schedules PEs in pairs, the change in the workload ratio between segments B and C requires increasing the optimal number of PEs for the Frontend from 12 to 14. It can be seen that the environment has a relatively minor impact on the workload in a single task.

Figure 12.(c) and Figure 12.(d) illustrate the ratios of Frontend to Backend processing times for three distinct executions of Bundle Adjustment (BA) using OpenMVG on the Sceaux Castle and DJI Mini2 datasets, with each BA optimizing different variables. Notably, the Frontend time proportion for the OpenMVG task is significantly greater when compared to the VINS series. Furthermore, within the OpenMVG framework, the optimization targets also influences the distribution of computational load to a certain degree.

Figure 13.(a) illustrates the instruction ratio related to the geometric perception Backend across different scenarios, with each bar representing the proportion of computational instructions assigned to a particular stage in relation to the total number of instructions. The *add* stage is executed within other stages.

In the VINS-Fusion task, the quantity of co-visible points between adjacent frames is greater than that observed in other tasks. This results in an increase in the merge command ratio from 1% to 10% and a decrease in *gcal* from 44% to 33%, indicating a reduction in the computational load on the Backend. Our IDEA-GP architecture manages the operations of PEs through configurable instructions, allowing the same PE to be adapted for various tasks. By utilizing diverse instruction flows, the computational workload of PEs can be allocated more effectively, thereby accommodating the requirements of different tasks. The hardware offers a high degree of flexibility.

Figure 13.(b) presents the computation times for both the Frontend and Backend, utilizing an identical configuration of processing elements (12 cores for each), while implementing our architecture across various tasks. It is apparent that the discrepancies in computation times between the Frontend and Backend on the FPGA are consistently aligned with those observed on the CPU across the different tasks. The imbalance in computation time between the Frontend and Backend results in the idling of PEs, which consequently leads to the inefficient use of computing resources and increased latency in optimization tasks. The implementation of our online workload allocation system facilitates the balancing of computation time and the reduction of latency. Further results will be presented in Section 9.3 and illustrated in Figure 14.

To sum up, the dynamic fluctuations in Frontend and Backend loads primarily stem from two sources. Primarily, the inherent nature of the algorithms themselves can cause significant workload variations. For instance, different vSLAM algorithms, SfM algorithms, or even the (BA) processes at various stages within SfM can all induce substantial changes in load distribution. Secondly, in extreme scenarios, drastic changes in the number of visual features or the number of co-visible points within the task environment can also alter the workload balance.

9.2 The Effects of Online Allocation

Throughout our experiments, we assess the computation times for a variety of tasks across multiple configurations. Figure 14 presents the computation times for both the Frontend and Backend, as well as the total computation time, under different allocations of PEs to the Frontend. Our predictions consistently demonstrate a close alignment with the optimal outcomes.

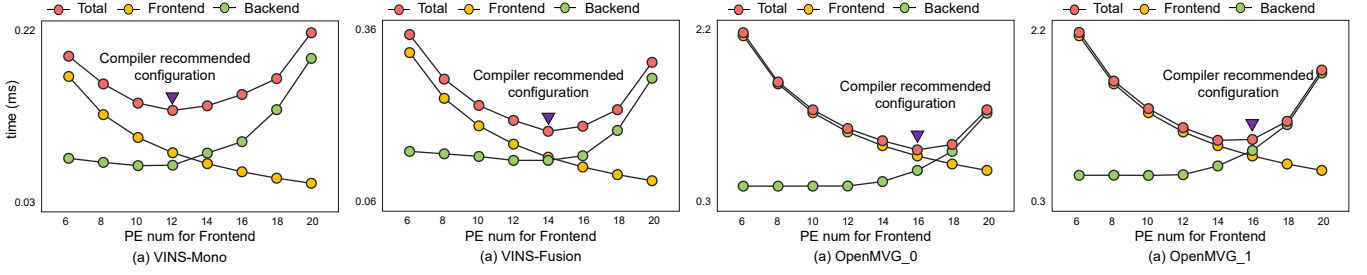


Figure 14: Changes in the Frontend-Backend PE ratio impact computation times at each phase, with the best allocation occurring when the workloads are well-balanced in the middle state.

From Figure 14, we observe that the shortest overall computation time does not always occur when the front-end and back-end computation times are equal. This is because our architecture processes these components in a pipeline, directly transmitting some Frontend results to the Back end. The differing data throughput capabilities of the two components can cause minor processing time discrepancies. However, by aligning the computational load, we can achieve near-optimal results, with a difference of less than 2% from the actual optimal computation time.

9.3 PE Utilization Analysis

The scalability of our architecture indicates that it is theoretically possible to enhance computational performance by increasing the number of PEs. However, experimental evidence shows that adding PEs does not lead to a linear improvement in the architecture’s computational capabilities.

Figure 14 illustrates the relationship between Backend computation time and the number of PEs for the VINS-Mono, VINS-Fusion, and OpenMVG tasks, highlighting the non-linear effects of increasing PEs on computational efficiency.

Take VINS-Mono for instance (Figure 14.(a), with the number of PEs in the Frontend increases, the number of PEs in the Backend decreases. When the PE number for the Frontend falls below 12, the PE for the Backend exceeds 12. Additionally, increasing the number of PEs for the Backend does not further decrease the latency of Backend processing. The overall computational power of Backward is limited by bandwidth constraints.

Nevertheless, enhancing the on-chip buffer bandwidth is relatively straightforward, indicating that our architecture is scalable and capable of supporting larger configurations.

9.4 Bandwidth Analysis

In terms of bandwidth considerations, we analyze the ratio of instruction bandwidth to data bandwidth across various tasks and datasets, as shown in Table 4.

The OpenMVG_0 and OpenMVG_1 represent two distinct optimization solutions for the computational tasks within OpenMVG.

The data consistently demonstrate that, in all analyzed scenarios, the bandwidth required for instructions constitutes less than 5% of that required for data. This result indicates that the demand for bandwidth associated with instructions is significantly lower than that for data.

9.5 Speedup and Resource Report

We implemented the 24-core IDEA-GP accelerator on the Zynq UltraScale+ ZCU102 platform, achieving utilization rates of 57.7% for LUTs, 15.6% for DSPs, and 38.4% for BRAMs.

As shown in Table 5, IDEA-GP achieves an average speedup of 7.5× over the Intel CPU and 19.7× over the ARM CPU on the VINS task. For the OpenMVG task, which involves more visual features and a sparser Jacobian matrix, IDEA-GP achieves a 16.4× speedup over the Intel CPU and a 41.6× speedup over the ARM CPU.

10 Related Works

A substantial body of literature concentrates on enhancing the applications of robotics, which can be categorized into two primary directions: dedicated design and general usage. Table 6 lists a portion of the works for comparison. Our architecture, with its consistent frontend and backend design, allows flexible allocation of PEs between the Frontend and Backend.

Table 4: Bandwidth Analysis

Mission	VINS-Mono			VINS-Fusion			OpenMVG_0		OpenMVG_1	
Dataset	MH_04	V11	V2	MH_04	V1	V2	SceauxCastle	DJI Mini2	SceauxCastle	DJI Mini2
#Residual num	1077	729	853	2073	1007	1521	13866	116820	13866	116820
Total Data Size (KB)	164.1	111.1	130.0	315.8	153.4	231.7	2112	17797	2112	17797
#On-chip Instruction num	8746	5231	6424	14200	6365	10243	73739	526244	69280	737460
#High-level Instruction num	790	703	706	3037	1162	2088	8634	94371	11554	125854
High-level Instruction Size (KB)	3.1	2.7	2.8	11.9	4.5	8.2	33.7	368.6	45.1	491.6
Instruction bandwidth ratio	1.9%	2.4%	2.2%	3.8%	2.9%	3.5%	1.6%	2.1%	2.1%	2.8%

Table 5: Time Report

Mission Dataset		VINS-Mono			VINS-Fusion			OpenMVG_0		OpenMVG_1	
		MH_04	V11	V2	MH_04	V1	V2	SceauxCastle	DJI Mini2	SceauxCastle	DJI Mini2
#Residual num		1077	729	853	2073	1007	1521	13866	116820	13866	116820
ARM	Frontend time (ms)	1.128	0.767	0.869	2.609	1.346	1.854	26.817	243.116	28.265	271.394
	Backend time (ms)	1.153	0.804	0.933	1.882	1.132	1.341	8.225	67.265	11.540	89.428
	overall time (ms)	2.281	1.571	1.802	4.491	2.478	3.195	35.042	310.381	39.805	360.822
Intel	Frontend time (ms)	0.487	0.312	0.345	0.910	0.475	0.650	9.672	89.669	10.093	95.998
	Backend time (ms)	0.498	0.337	0.389	0.683	0.346	0.510	3.414	27.889	4.758	37.787
	overall time (ms)	0.985	0.649	0.734	1.593	0.821	1.160	13.086	117.558	14.851	133.785
OURS	Frontend time (ms)	0.083	0.056	0.066	0.138	0.067	0.100	0.810	6.834	0.811	7.810
	Backend time (ms)	0.069	0.045	0.050	0.132	0.063	0.096	0.649	5.596	0.873	7.355
	overall time (ms)	0.129	0.099	0.109	0.183	0.114	0.148	0.880	7.277	0.997	8.242
speedup to ARM		17.6×	15.9×	16.5×	25.0×	21.7×	21.6×	39.8×	42.7×	39.9×	43.8×
speedup to Intel		7.6×	6.6×	6.7×	8.9×	7.2×	7.8×	14.9×	16.1×	14.9×	16.2×

Table 6: Comparison of Related Works

Works	Task		Frontend	Backend	Generation Architecture
	vSLAM	SfM			
π -B[36]	×	✓	fixed	fixed	fixed
Navion[45]	×	×	fixed	fixed	fixed
Archytas[26]	✓	×	adjustable	fixed	re-generation
Orianna[17]	✓	✓	adjustable	adjustable	re-generation
Ours	✓	✓	adjustable	adjustable	online instruction-driven

Dedicated Robot Applications Hardware. Previous research has primarily concentrated on enhancing various robot applications including localization and mapping [2, 5, 15, 16, 19, 23, 27, 36, 45, 51, 55, 56], planning [3, 18, 22, 24, 29–31, 41, 42], learning [1, 8, 9, 13, 52], and control [20, 35, 53], typically by proposing dedicated hardware that accelerates either a single specific algorithm or a limited range of similar applications. However, even minor variations in the problem domain can lead to the failure of these dedicated accelerators, rendering the computational units unsuitable for reuse across different tasks. In contrast, IDEA-GP is not a dedicated accelerator like those described in earlier studies; rather, it employs a set of general computational units designed to accommodate a variety of algorithms and tasks through scheduled instructions. Our architecture enables the compiler to dynamically allocate the number of PEs based on the workload of the geometric perception task, thereby enhancing efficiency, universality, and scalability.

Hardware Generation Architecture. The architecture of hardware generation differs from fixed architectures in that it is reconstructed in response to varying tasks [32, 34, 39, 49]. Previous research has predominantly focused on non-visual tasks, while some studies have attempted to create data flow graphs (DFGs) or factor graphs as a common abstraction for generating hardware accelerators tailored to optimization algorithms typically employed in geometric perception [17, 26]. For instance, Archytas [26] introduced a DFG-based generation framework for SLAM. However, this framework necessitates the regeneration of hardware for different applications and lacks data transfer instructions for Double Data Rate (DDR) memory, which constrains the scalability of the target problem, such as SfM in embodied artificial intelligence. Additionally, ORIANNA [17] proposed a general representation of robot position and orientation using Lie Algebra to generate accelerators for robot applications that incorporate multiple optimization-based

algorithms. While factor graphs are commonly utilized for optimizing the ego pose of robots, they are less applicable than Bundle Adjustment (BA) for optimizing spatial feature points. Furthermore, ORIANNA also requires hardware regeneration in response to fluctuating workload conditions across various tasks. In contrast, our architecture, which is based on instructions and PEs with a uniform structure, allows IDEA-GP to adapt to different workloads across diverse applications without the need for hardware reconstruction.

IDEA-GP represents a significant leap forward from prior work by enabling online configuration of the accelerator through an instruction-driven approach. This offers two key benefits: First, IDEA-GP can switch seamlessly between different algorithms without needing re-generation. Second, its online resource and workload allocation ensure optimal computational speed for each algorithm and scenario. Moreover, eliminating hardware modifications makes ASIC deployment a viable option.

11 Conclusion

This paper presents an instruction-driven architecture designed for general robot geometric perception, incorporating efficient online workload allocation. The proposed framework, IDEA-GP, employs instructions to orchestrate a set of basic computational units that we have developed. This approach enables efficient online hardware resource allocation for various tasks without hardware regeneration. Furthermore, we assess the performance of different workloads across various tasks on the ZCU102 board, achieving an average speedup of 7.5× over the Intel CPU and 19.7× over the ARM CPU in SLAM tasks, and a 16.4× speedup over the Intel CPU and 41.6× over the ARM CPU in SfM tasks.

Acknowledgments

This research was supported by National Natural Science Foundation of China (No.62203257, 62325405, 62031017), Tsinghua University Initiative Scientific Research Program, Tsinghua-Efort Joint Research Center for EAI Computation and Perception, Beijing National Research Center for Information Science, Technology (BN-Rist), Beijing Innovation Center for Future Chips, and State Key laboratory of Space Network and Communications.

References

- [1] Ameer M. S. Abdelhadi, Eugene Sha, Ciaran Bannon, Hendrik Steenland, and Andreas Moshovos. 2021. Noema: Hardware-Efficient Template Matching for Neural Population Pattern Detection. In *MICRO '21: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [2] Bahar Asgari, Ramiyad Hadidi, Nima Shoghi Ghaleshahi, and Hyesoon Kim. 2020. PISCES: Power-Aware Implementation of SLAM by Customizing Efficient Sparse Algebra. In *DAC '20: Proceedings of the 57th ACM/EDAC/IEEE Design Automation Conference*.
- [3] M. Bakhshalipour, S.B. Ehsani, M. Qadri, D. Guri, M. Likhachev, and P.B. Gibbons. 2022. RACOD: algorithm/hardware co-design for mobile robot path planning. In *ISCA '22: Proceedings of the 49th Annual International Symposium on Computer Architecture*.
- [4] Mohammad Bakhshalipour and Phillip B. Gibbons. 2024. Tartan: Microarchitecting a Robotic Processor. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.
- [5] Konstantinos Boikos and Christos-Savvas Bouganis. 2016. Semi-dense SLAM on an FPGA SoC. In *2016 26th International Conference on Field Programmable Logic and Applications (FPL)*.
- [6] Michael Burri, Janosch Nikolic, Pascal Gohl, Thomas Schneider, Joern Rehder, Sammy Omari, Markus W. Achtelik, and Roland Siegwart. 2016. The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research* (2016). doi:10.1177/0278364915620033 arXiv:https://arxiv.org/abs/1601.07159
- [7] Cesar Cadena, Luca Carlone, Henry Carrillo, Yaser Latif, Davide Scaramuzza, José, Neira, Ian Reid, and John J. Leonard. 2016. Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics* (2016), 1309–1332. Issue 6.
- [8] Ningyuan Cao, Muya Chang, and Arijit Raychowdhury. 2019. 14.1 A 65nm 1.1-to-9.1TOPS/W Hybrid-Digital-Mixed-Signal Computing Platform for Accelerating Model-Based and Model-Free Swarm Robotics. In *2019 IEEE International Solid-State Circuits Conference - (ISSCC)*.
- [9] C. Chen, X. Zou, H. Shao, Y. Li, and K. Li. 2023. Point Cloud Acceleration by Exploiting Geometric Similarity. In *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [10] F. Chen, R. Ying, J. Xue, F. Wen, and P. Liu. 2023. ParallelNN: A Parallel Octree-based Nearest Neighbor Search Accelerator for 3D Point Clouds. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*.
- [11] Shoubin Chen, Hao Ma, Changhui Jiang, Baoding Zhou, Weixing Xue, Zhenzhong Xiao, and Qingquan Li. 2022. NDT-LOAM: A Real-Time Lidar Odometry and Mapping With Weighted NDT and LFA. *IEEE Sensors Journal* (2022), 3660–3671. Issue 4.
- [12] David R Cox and E Joyce Snell. 1968. A general definition of residuals. *Journal of the Royal Statistical Society: Series B (Methodological)* 30, 2 (1968), 248–265.
- [13] Yu Feng, Boyuan Tian, Tiancheng Xu, Paul Whatmough, and Yuhao Zhu. 2020. Mesorasi: Architecture Support for Point Cloud Analytics via Delayed-Aggregation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*.
- [14] Yu Feng, Paul Whatmough, and Yuhao Zhu. 2019. In *MICRO '52: Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*.
- [15] Yiming Gan, Bo Yu, Boyuan Tian, Leimeng Xu, Wei Hu, Shaoshan Liu, Qiang Liu, Yanjun Zhang, Jie Tang, and Yuhao Zhu. 2021. Eudoxus: Characterizing and Accelerating Localization in Autonomous Machines. (2021).
- [16] Quentin Gautier, Alric Althoff, and Ryan Kastner. 2019. FPGA Architectures for Real-time Dense SLAM. In *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, Vol. 2160-052X. 83–90. doi:10.1109/ASAP.2019.00-25
- [17] Yuhui Hao, Yiming Gan, Bo Yu, Qiang Liu, Yinhe Han, Zishen Wan, and Shaoshan Liu. 2024. ORIANNA: An Accelerator Generation Framework for Optimization-based Robotic Applications. In *ASPLOS '24: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.
- [18] Y. Hao, Y. Gan, B. Yu, Q. Liu, S.-S. Liu, and Y. Zhu. 2023. BLITZCRANK: Factor Graph Accelerator for Motion Planning. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*.
- [19] Yuhui Hao, Bo Yu, Qiang Liu, Shaoshan Liu, and Yuhao Zhu. 2022. Factor Graph Accelerator for LiDAR-Inertial Odometry (Invited Paper). In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (San Diego, California) (ICCAD '22)*. Association for Computing Machinery, New York, NY, USA, Article 103, 7 pages. doi:10.1145/3508352.3561112
- [20] Yuhui Hao, Bo Yu, Qiang Liu, and Shao-Shan Liu. 2023. FGLQR: Factor Graph Accelerator of LQR Control for Autonomous Machines. arXiv:2308.02768 [cs.AR] https://arxiv.org/abs/2308.02768
- [21] Jueun Jung, Seungbin Kim, Bokyoung Seo, Wuyoung Jang, Sangho Lee, Jeongmin Shin, Donghyeon Han, and Kyuho Jason Lee. 2024. 20.6 LSPU: A Fully Integrated Real-Time LiDAR-SLAM SoC with Point-Neural-Network Segmentation and Multi-Level KNN Acceleration. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*.
- [22] Atsutake Kosuge and Takashi Oshima. 2020. A 1200×1200 8-Edges/Vertex FPGA-Based Motion-Planning Accelerator for Dual-Arm-Robot Manipulation Systems. In *2020 IEEE Symposium on VLSI Circuits*.
- [23] Ziyun Li, Yu Chen, Luyao Gong, Lu Liu, Dennis Sylvester, David Blaauw, and Hun-Seok Kim. 2019. An 879GOPS 243mW 80fps VGA Fully Visual CNN-SLAM Processor for Wide-Range Autonomous Exploration. In *2019 IEEE International Solid-State Circuits Conference - (ISSCC)*. 134–136. doi:10.1109/ISSCC.2019.8662397
- [24] Shiqi Lian, Yinhe Han, Xiaoming Chen, Ying Wang, and Hang Xiao. 2018. Dadu-P: a scalable accelerator for robot motion planning in a dynamic environment. In *DAC '18: Proceedings of the 55th Annual Design Automation Conference*.
- [25] Runze Liu, Jianlei Yang, Yiran Chen, and Weisheng Zhao. 2019. eSLAM: An Energy-Efficient Accelerator for Real-Time ORB-SLAM on FPGA Platform. In *DAC '19: Proceedings of the 56th Annual Design Automation Conference 2019*.
- [26] Weizhuang Liu, Bo Yu, Yiming Gan, Qiang Liu, Jie Tang, Shaoshan Liu, and Yuhao Zhu. 2021. Archytas: A framework for synthesizing and dynamically optimizing accelerators for robotic localization. In *MICRO '21: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [27] Dipan Kumar Mandal, Srivatsava Jandhyala, Om J Omer, Gurpreet S Kalsi, Biji George, Gopi Neela, Santhosh Kumar Rethinagiri, Sreenivas Subramoney, Lance Hacking, Jim Radford, Eagle Jones, Bellappa Kuttanna, and Hong Wang. 2019. Visual Inertial Odometry At the Edge: A Hardware-Software Co-design Approach for Ultra-low Latency and Power. In *2019 Design, Automation and Test in Europe Conference and Exhibition (DATE)*.
- [28] Pierre Moulon, Pascal Monasse, Romuald Perrot, and Renaud Marlet. 2017. Open-MVG: Open Multiple View Geometry. In *Reproducible Research in Pattern Recognition*, Bertrand Kerautret, Miguel Colom, and Pascal Monasse (Eds.). Springer International Publishing, Cham, 60–74.
- [29] Sean Murray, Will Floyd-Jones, George Konidaris, and Daniel J. Sorin. 2019. A Programmable Architecture for Robot Motion Planning Acceleration. In *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*.
- [30] Sean Murray, William Floyd-Jones, Ying Qi, George Konidaris, and Daniel J. Sorin. 2016. The microarchitecture of a real-time robot motion planning accelerator. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–12. doi:10.1109/MICRO.2016.7783748
- [31] Sean Murray, Will Floyd-Jones, Ying Qi, Daniel J. Sorin, and George Dimitri Konidaris. 2016. Robot Motion Planning on a Chip. In *Robotics: Science and Systems*. https://api.semanticscholar.org/CorpusID:15540359
- [32] S.M. Neuman, R. Ghosal, T. Bourgeat, B. Plancher, and V.J. Reddi. 2023. RoboShape: Using Topology Patterns to Scalably and Flexibly Deploy Accelerators Across Robots. In *ISCA '23: Proceedings of the 50th Annual International Symposium on Computer Architecture*.
- [33] Sabrina M. Neuman, Brian Plancher, Thomas Bourgeat, Thierry Tambe, Srinivas Devadas, and Vijay Janapa Reddi. 2021. Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology. In *ASPLOS 2021: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [34] Sabrina M Neuman, Brian Plancher, Thomas Bourgeat, Thierry Tambe, Srinivas Devadas, and Vijay Janapa Reddi. 2021. Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology. In *ASPLOS '21: 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [35] L. Patterson, D. Pigorovsky, B. Dempsey, N. Lazarev, A. Shah, C. Steinhoff, A. Bruno, J. Hu, and C. Delimitrou. 2022. HiveMind: a hardware-software system stack for serverless edge swarms. In *ISCA '22: Proceedings of the 49th Annual International Symposium on Computer Architecture*.
- [36] Shuzhen Qin, Qiang Liu, Bo Yu, and Shaoshan Liu. 2019. pi-BA: Bundle Adjustment Acceleration on Embedded FPGAs with Co-observation Optimization. In *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*.
- [37] Tong Qin, Shaozu Cao, Jie Pan, and Shaojie Shen. 2019. A General Optimization-based Framework for Global Pose Estimation with Multiple Sensors. arXiv:1901.03642 [cs.CV] https://arxiv.org/abs/1901.03642
- [38] Tong Qin, T. (Qin, Peiliang) Li, PL (Li, and Shaojie) Shen, SJ (Shen. 2018. VINS-Mono: a robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics* (2018), 1004–1020. Issue 4.
- [39] Jacob Sacks, Divya Mahajan, Richard C. Lawson, and Hadi Esmaeilzadeh. 2018. Robox: an end-to-end solution to accelerate autonomous control in robotics. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*.
- [40] Johannes L. Schonberger and Jan Michael Frahm. 2016. Structure-from-Motion Revisited. In *IEEE Conference on Computer Vision and Pattern Recognition*. 4104–4113.
- [41] Deval Shah and Tor M. Aamodt. 2024. Collision Prediction for Robotics Accelerators. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.

- [42] D. Shah, N. Yang, and T.M. Aamodt. 2023. Energy-Efficient Realtime Motion Planning. In *ISCA '23: Proceedings of the 50th Annual International Symposium on Computer Architecture*.
- [43] Joan Sola, Jeremie Deray, and Dinesh Atchuthan. 2018. A micro Lie theory for state estimation in robotics. *arXiv preprint arXiv:1812.01537* (2018).
- [44] S.D. Spetalnick, A.S. Lele, B. Crafton, M. Chang, S. Ryu, J.-H. Yoon, Z. Hao, A. Ansari, W.-S. Khwa, Y.-D. Chih, M.-F. Chang, and A. Raychowdhury. 2024. A 40nm VLIW Edge Accelerator with 5MB of 0.256pJ/b RRAM and a Localization Solver for Bristle Robot Surveillance. In *2024 IEEE International Solid-State Circuits Conference (ISSCC)*.
- [45] A. Suleiman, Zhengdong Zhang, L. Carlone, S. Karaman, and V. Sze. 2019. Navion: A 2-mW Fully Integrated Real-Time Visual-Inertial Odometry Accelerator for Autonomous Navigation of Nano Drones. *IEEE Journal of Solid-State Circuits* (2019), 1106–1119. Issue 4.
- [46] Ali Tourani, Hriday Bavle, Jose Luis Sanchez-Lopez, and Holger Voos. 2022. Visual SLAM: What Are the Current Trends and What to Expect? *Sensors* 22, 23 (Nov. 2022), 9297. doi:10.3390/s22239297
- [47] Zishen Wan, Ashwin Lele, Bo Yu, Shaoshan Liu, Yu Wang, Vijay Janapa Reddi, Cong Hao, and Arijit Raychowdhury. 2022. Robotic Computing on FPGAs: Current Progress, Research Challenges, and Opportunities. arXiv:2205.07149 [cs.RO] <https://arxiv.org/abs/2205.07149>
- [48] Zishen Wan, Bo Yu, Thomas Yuang Li, Jie Tang, Yuhao Zhu, Yu Wang, Arijit Raychowdhury, and Shaoshan Liu. 2021. A Survey of FPGA-Based Robotic Computing. *IEEE Circuits and Systems Magazine* 21, 2 (2021), 48–74. doi:10.1109/MCAS.2021.3071609
- [49] Jian Weng, Sihao Liu, Vidushi Dadu, Zhengrong Wang, Preyas Shah, and Tony Nowatzki. 2020. DSAGEN: Synthesizing Programmable Spatial Accelerators. In *ISCA '20: Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture*.
- [50] Jong-Hyeok Yoon and Arijit Raychowdhury. 2020. 31.1 A 65nm 8.79TOPS/W 23.82mW Mixed-Signal Oscillator-Based NeuroSLAM Accelerator for Applications in Edge Robotics. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*.
- [51] Jae-Sung Yoon, Jeong-Hyun Kim, Hyo-Eun Kim, Won-Young Lee, Seok-Hoon Kim, Kyusik Chung, Jun-Seok Park, and Lee-Sup Kim. 2013. A Unified Graphics and Vision Processor With a 0.89 /spl mu/W/fps Pose Estimation Engine for Augmented Reality. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* (2013), 206–216. Issue 2.
- [52] Jie-Fang Zhang and Zhengya Zhang. 2021. Point-X: A Spatial-Locality-Aware Architecture for Energy-Efficient Graph-Based Point-Cloud Deep Learning. In *MICRO '21: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [53] Pei(peizhang@iastate.edu) Zhang, Aaron(ajmills@iastate.edu) Mills, Joseph(zambreno@iastate.edu) Zambreno, and Phillip H.(phjones@iastate.edu) Jones. 2017. The design and integration of a software configurable and parallelized coprocessor architecture for LQR control. *J. Parallel and Distrib. Comput.* (2017), 121–131. Issue No.C.
- [54] Suquan Zhang, Jincheng Yu, Yuanfan Xu, and Yu Wang. 2023. UAV Swarm Planning accelerator on FPGA with low latency and fixed-point L-BFGS Quasi-Newton solver. In *2023 International Conference on Field Programmable Technology (ICFPT)*. IEEE, 284–285.
- [55] Zhe Zhang, Shaoshan Liu, Grace Tsai, Hongbing Hu, Chen-Chi Chu, and Feng Zheng. 2018. PIRVS: An Advanced Visual-Inertial SLAM System with Flexible Sensor Fusion and Hardware Co-Design. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*.
- [56] Zhengdong Zhang, Amr Suleiman, and Luca Carlon. 2017. Visual-inertial odometry on chip: An algorithm-and-hardware co-design approach. In *2017 Robotics: Science and Systems, RSS 2017*.